# NEXTRADE

# STOCK TRADING APP

STEPS OF PROJECT EXICUTION

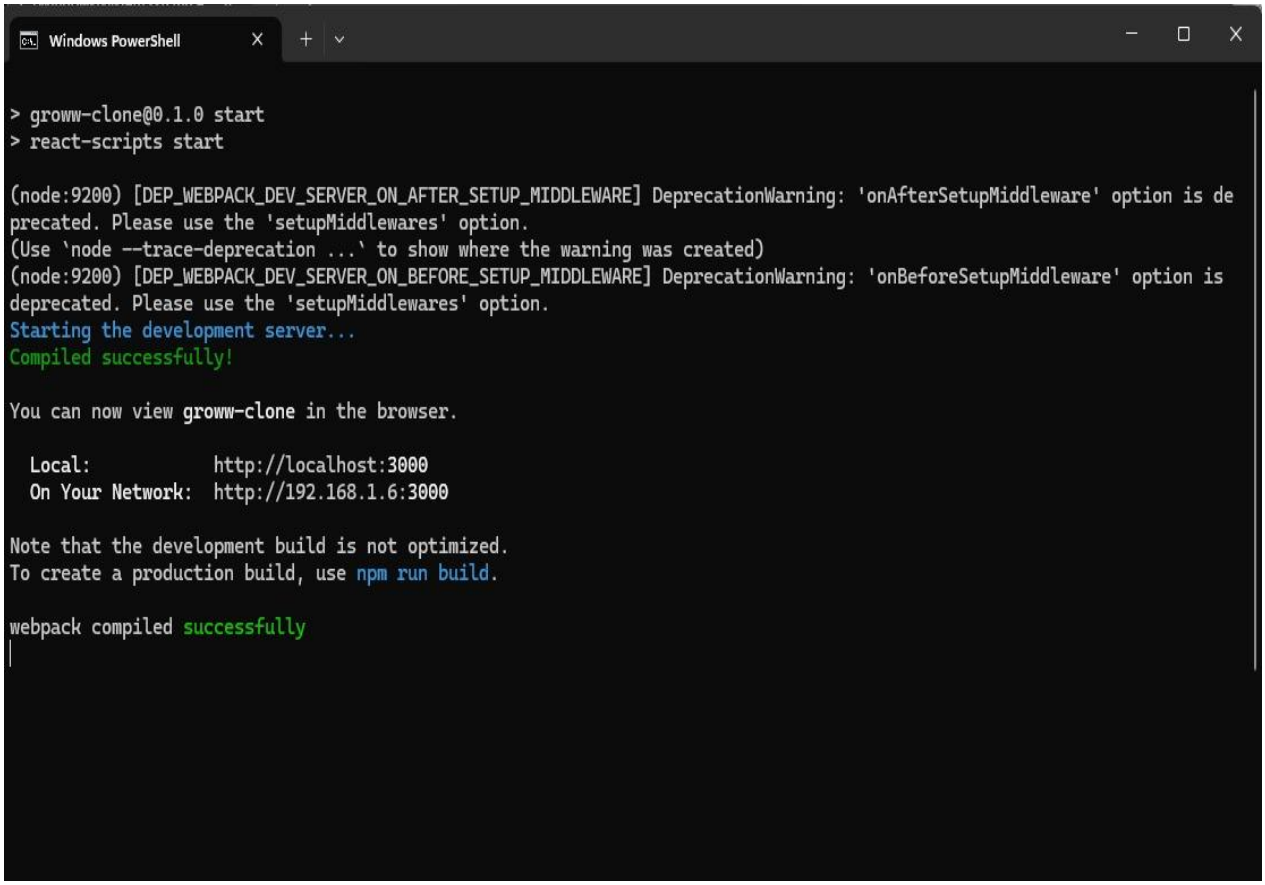
STEP – 1

```
C:\Windows\system32\cmd.e:

(c) Microsoft Corporation. All rights reserved.

D:\Downloads\STOCK-TRADING-APP-main>cd backend

D:\Downloads\STOCK-TRADING-APP-main\backend>npm run dev

> backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (7) from .env -- tip: 👮‍♀️ agentic secret storage: https://dotenvx.com/as2
Server running in development mode on port 5000
MongoDB Connected: stock-trading (Primary)
MongoDB Connected: stockDB (Secondary)
(node:17964) [MONGOOSE] Warning: mongoose: Duplicate schema index on {"email":1} for model "User". This is often due to
declaring an index using both "index: true" and "schema.index()". Please remove the duplicate index definition.
(Use `node --trace-warnings ...` to show where the warning was created)
Incoming Request: OPTIONS /api/auth/login
Incoming Request: POST /api/auth/login
Incoming Request: OPTIONS /api/auth/login
Incoming Request: POST /api/auth/login
Incoming Request: OPTIONS /api/auth/register
Incoming Request: POST /api/auth/register
Incoming Request: OPTIONS /api/orders/buy
Incoming Request: POST /api/orders/buy
```

**Windows Command Prompt (cmd)** window where a **Node.js backend server** for a **Stock Trading App** is running. I'll explain each part step-by-step so it's easier to understand.

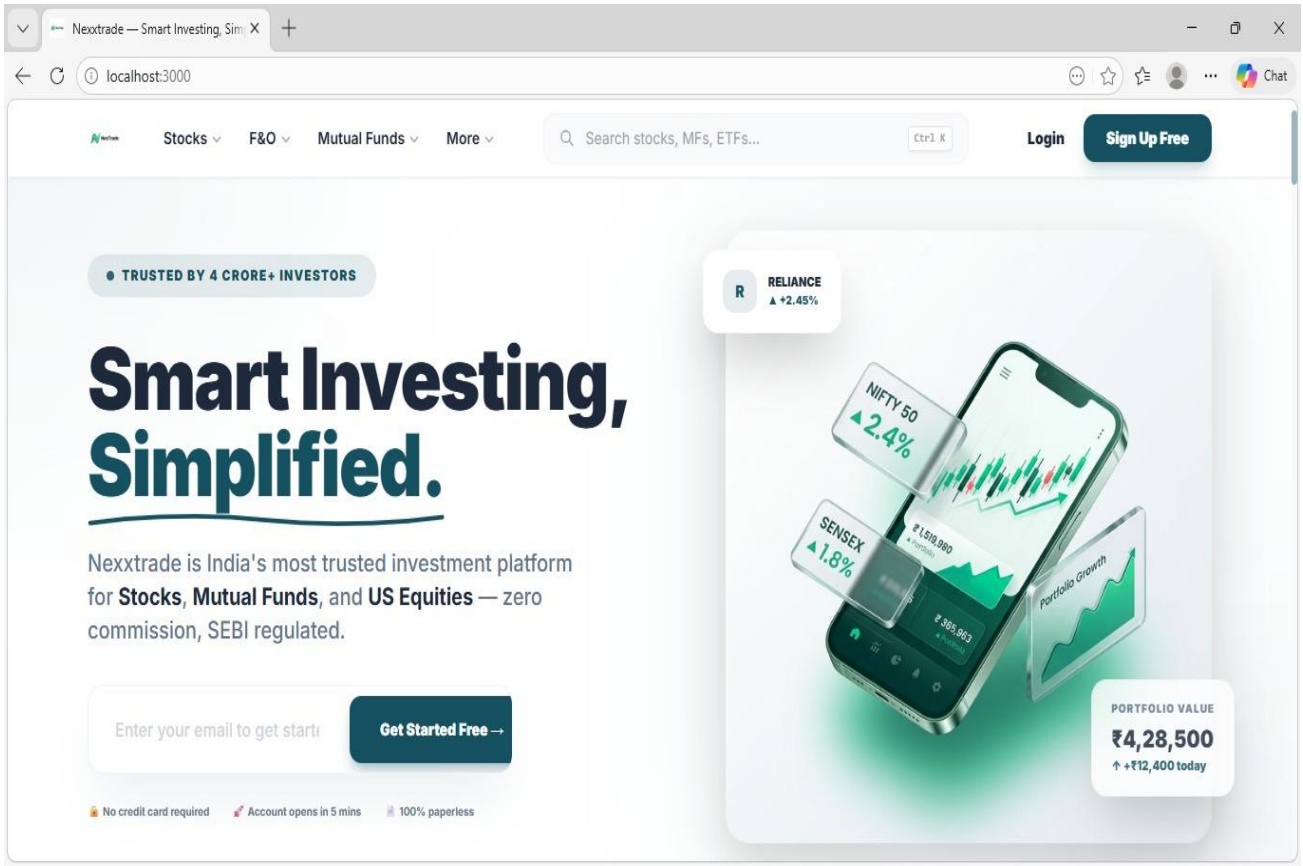
1. Command Prompt Header
2. Project Folder Navigation
3. Starting the Development Server
4. Nodemon Information
5. Environment Variables
6. Server Started
7. MongoDB Database Connection
8. Warning Message
9. API Requests Log

STEP – 2



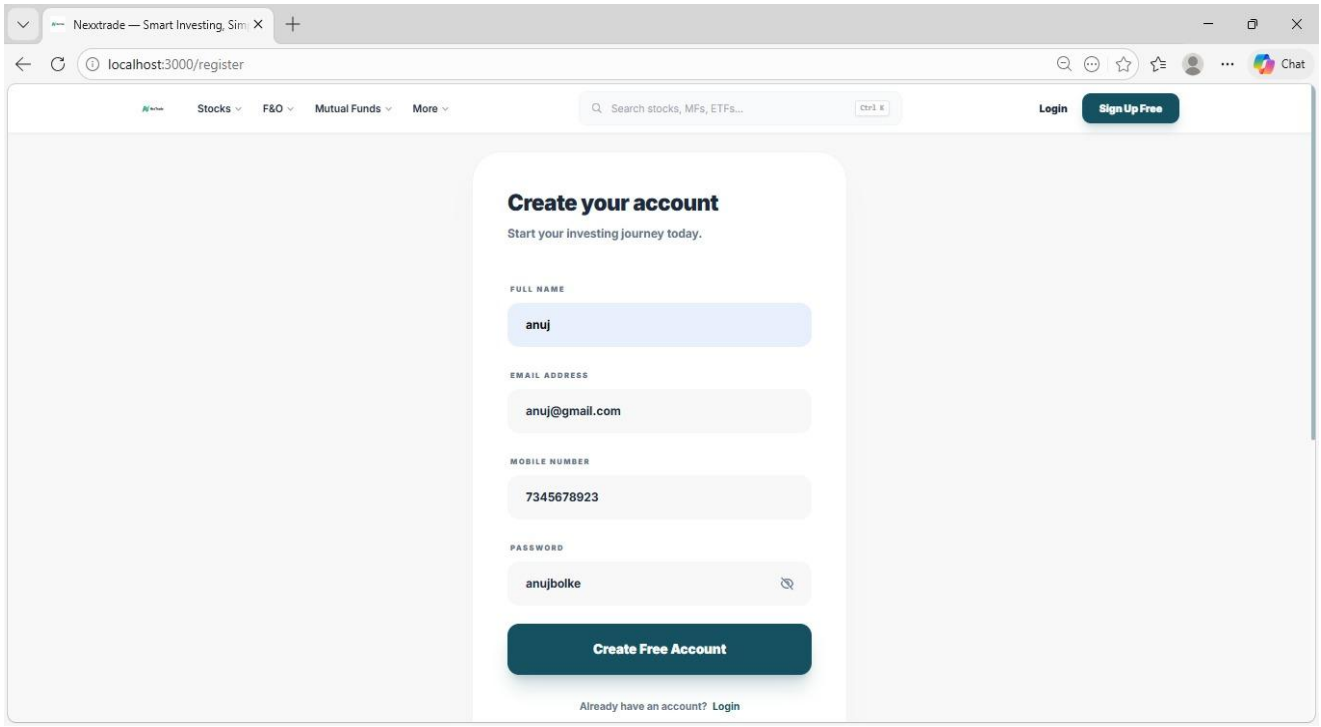

This screenshot shows **Windows PowerShell** running the **frontend of a React application** called *groww-clone*. The project is started using the command react-scripts start, which launches the development server. Some **webpack deprecation warnings** are displayed, indicating that certain middleware options are outdated, but they do not stop the application from running. The development server starts successfully and the project is **compiled by webpack without errors**. The application can now be viewed in the browser at **http://localhost:3000** on the local machine. It also provides a **network URL (192.168.1.6:3000)** so other devices on the same network can access it. A note indicates that this is a **development build and not optimized for production**. Overall, this page confirms that the **React frontend server is running successfully and ready to use**.

STEP – 3



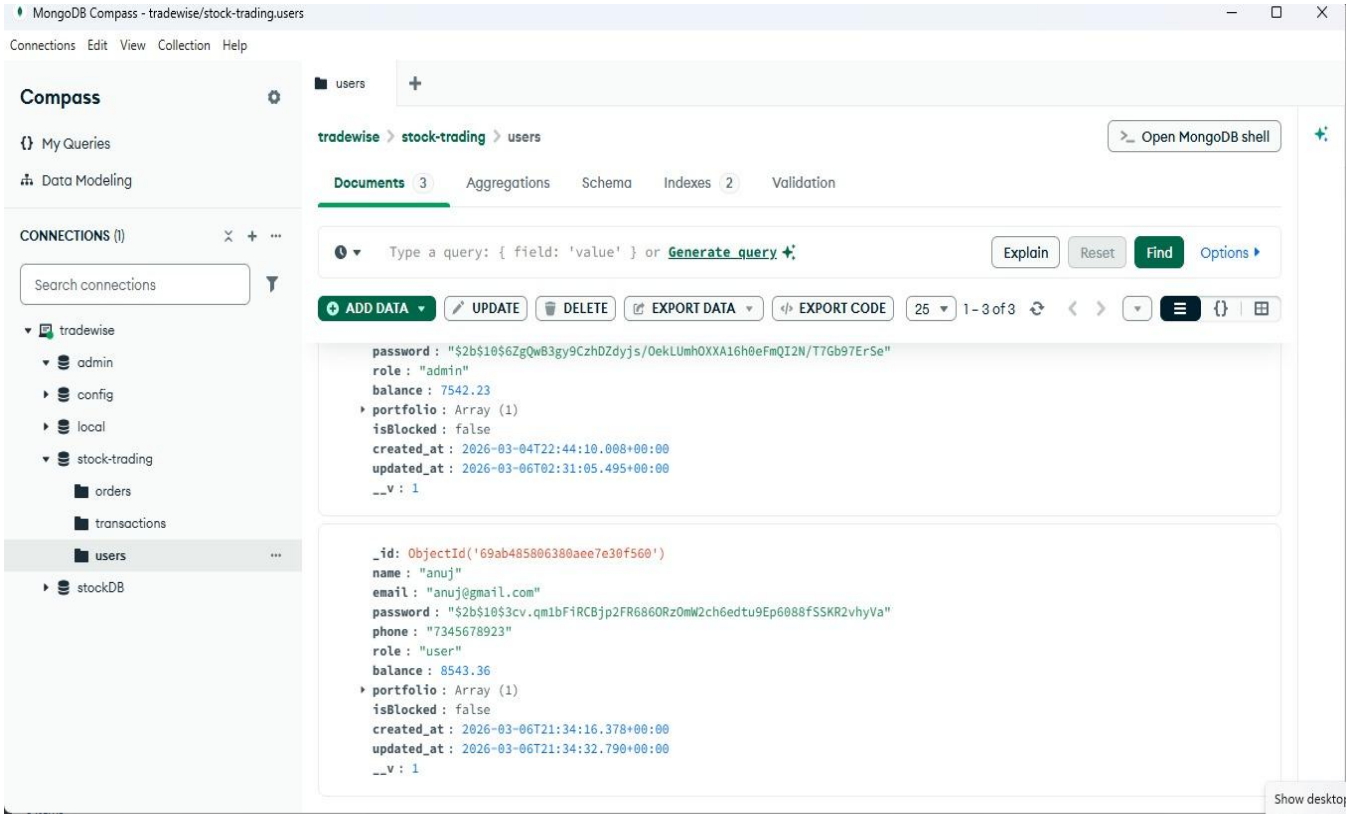

This screenshot shows the **homepage of a stock trading web application** running on **localhost:3000**, which means the project is running on a local development server. The platform is named **Nexxtrade** and is designed for investing in **stocks, mutual funds, and US equities**. At the top, the navigation bar includes options like **Stocks, F&O, Mutual Funds**, a search bar, and **Login / Sign Up** buttons. The main heading **"Smart Investing, Simplified."** introduces the platform as an easy investment solution. A **Get Started Free** button allows users to begin the registration process. On the right side, a visual graphic shows stock market indicators like **NIFTY 50, SENSEX, and portfolio growth**

STEP – 4



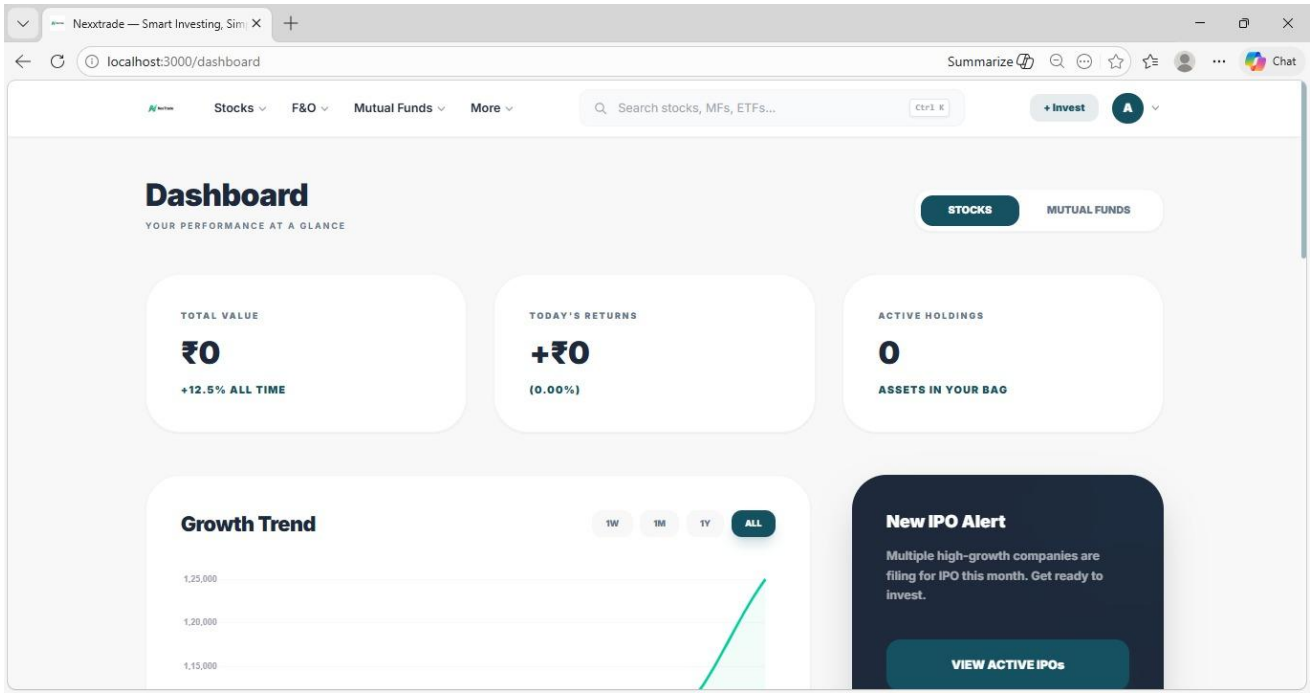

This screenshot shows the **user registration page** of the **Nexxtrade stock trading web application** running on **localhost:3000/register**. The page allows new users to create an account to start their investing journey. It contains a form where users enter details such as **Full Name, Email Address, Mobile Number, and Password**. After filling in the details, users can click the **"Create Free Account"** button to register. At the bottom, there is also an option for existing users to **login if they already have an account**. This page is used to **store new user information in the system before accessing the platform**.

STEP – 5



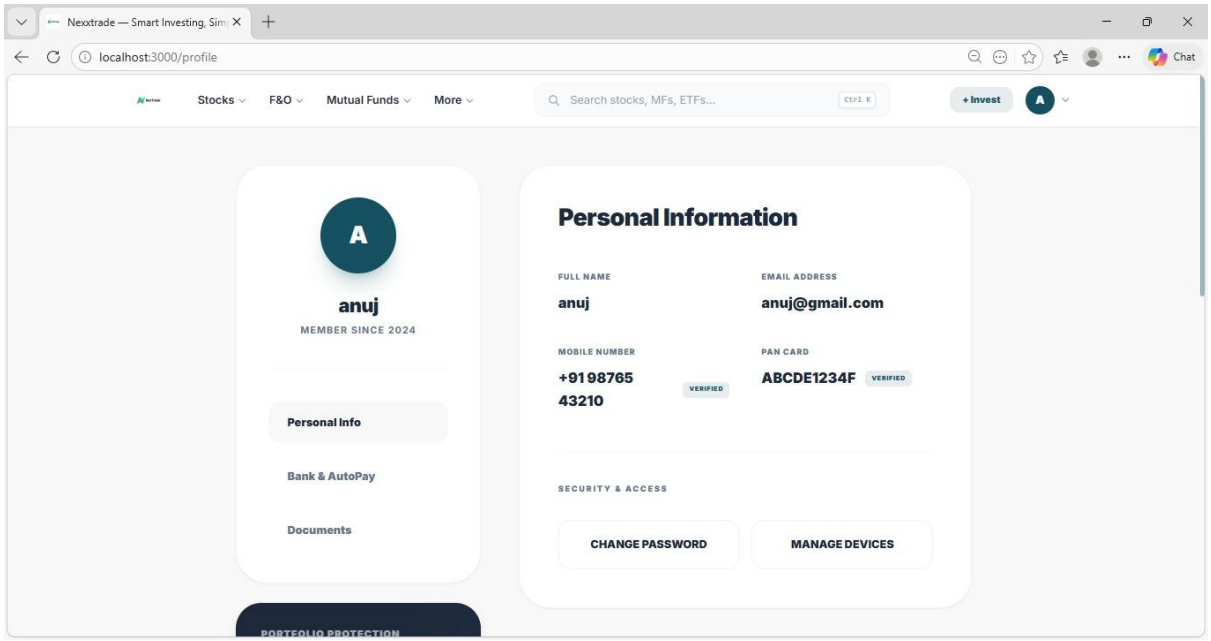

This screenshot shows **MongoDB Compass**, which is a graphical tool used to view and manage a **MongoDB** database. The screen displays the **users collection** inside the **stock-trading database**. It contains stored user records created from the registration form in the application. Each document shows details such as **name, email, phone number, password (encrypted), role, and account balance**. The data confirms that the **user registration information has been successfully saved in the database**. Developers use this interface to **view, update, delete, or export database records** for the application.

STEP - 6



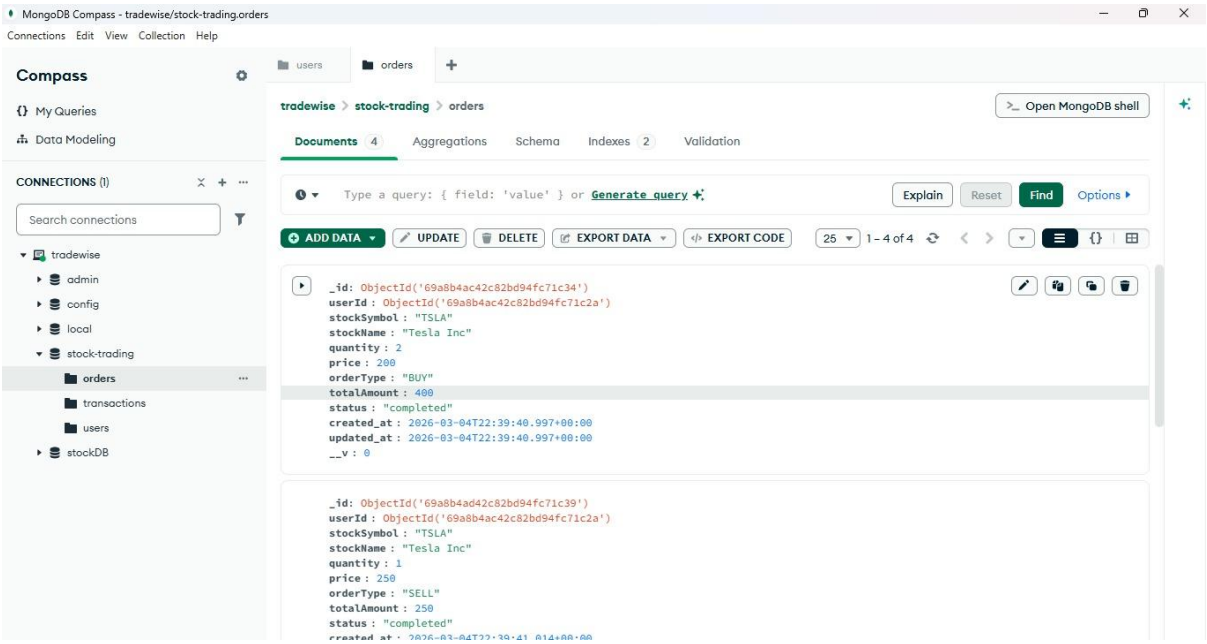

This page is a clean investment dashboard designed to give users a quick overview of their portfolio performance. The top navigation bar provides easy access to key sections like Stocks, F&O, Mutual Funds, and includes a search bar for quickly finding assets. The summary cards (Total Value, Today's Returns, Active Holdings) use a card-based UI to highlight the most important financial metrics at a glance. The Growth Trend chart visually represents portfolio performance over time with filter options (1W, 1M, 1Y, All) for quick analysis. On the right side, the IPO alert card acts as a call-to-action, encouraging users to explore new investment opportunities. The layout follows a clear visual hierarchy, with large headings, spacing, and soft shadows to improve readability. Overall, the interface uses minimalist design, rounded components, and balanced whitespace to create a modern fintech user experience.

STEP – 7




This page represents the user profile section of the Nextrade investment platform, designed to manage personal and security information. The top navigation bar remains consistent with the dashboard, maintaining usability and easy access to stocks, mutual funds, and investment options. On the left side, a profile card displays the user avatar, name, and membership date, creating a clear identity section. Below it, a vertical menu (Personal Info, Bank & AutoPay, Documents) helps users navigate different account settings easily. The right side contains a Personal Information card that organizes details like name, email, mobile number, and PAN card in a structured layout. Verification badges next to sensitive data improve trust and security perception for users. At the bottom, Security & Access actions such as Change Password and Manage Devices provide quick account control. Overall, the page follows a two-column layout, card-based design, and soft shadows, ensuring clarity, easy navigation, and a modern fintech UI experience

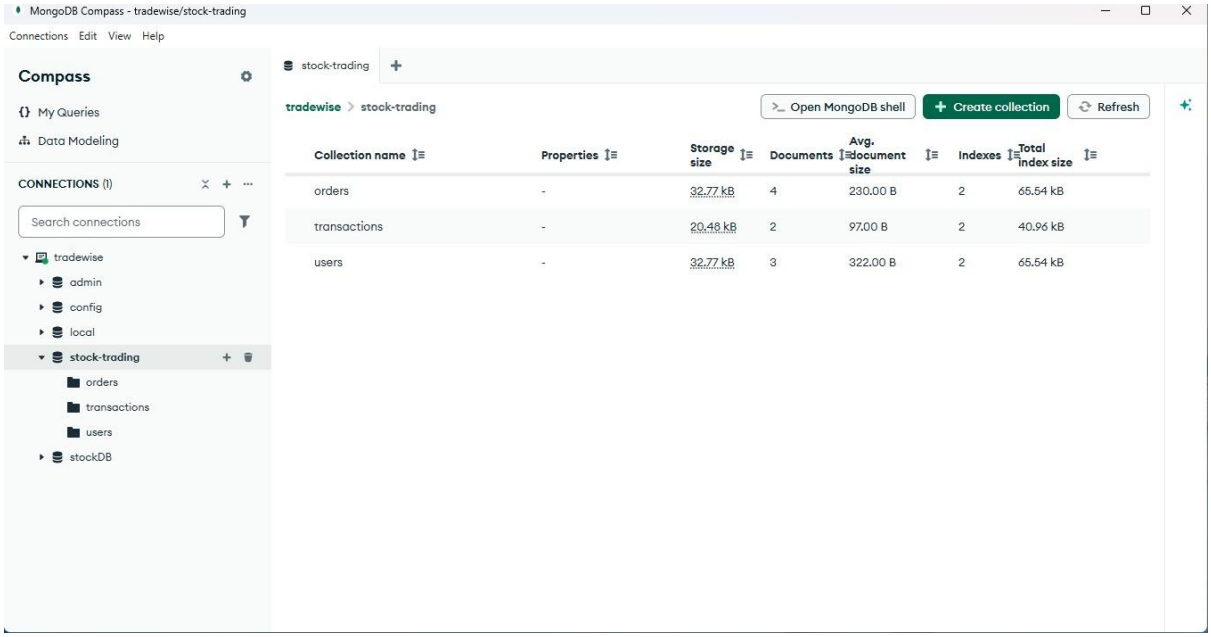
STEP – 8




MongoDB Orders Collection – UI/UX & Data Breakdown

This screen shows the MongoDB Compass interface where the orders collection inside the stock-trading database is being viewed. The left sidebar displays the database structure, including collections like users, orders, and transactions, making navigation between datasets simple. The main workspace shows the orders collection under the Documents tab, where stored order records are listed in JSON format. Each document represents a stock trade order containing fields such as stockSymbol, stockName, quantity, price, orderType (BUY/SELL), and totalAmount. The interface also shows userId references, linking each order to a specific user in the users collection. The status field (completed) and timestamps like created_at and updated_at help track order lifecycle and history. Action buttons like Add Data, Update, Delete, and Export allow developers to manage records efficiently. Overall, this interface provides a structured database management view for tracking stock trading transactions in the application backend.

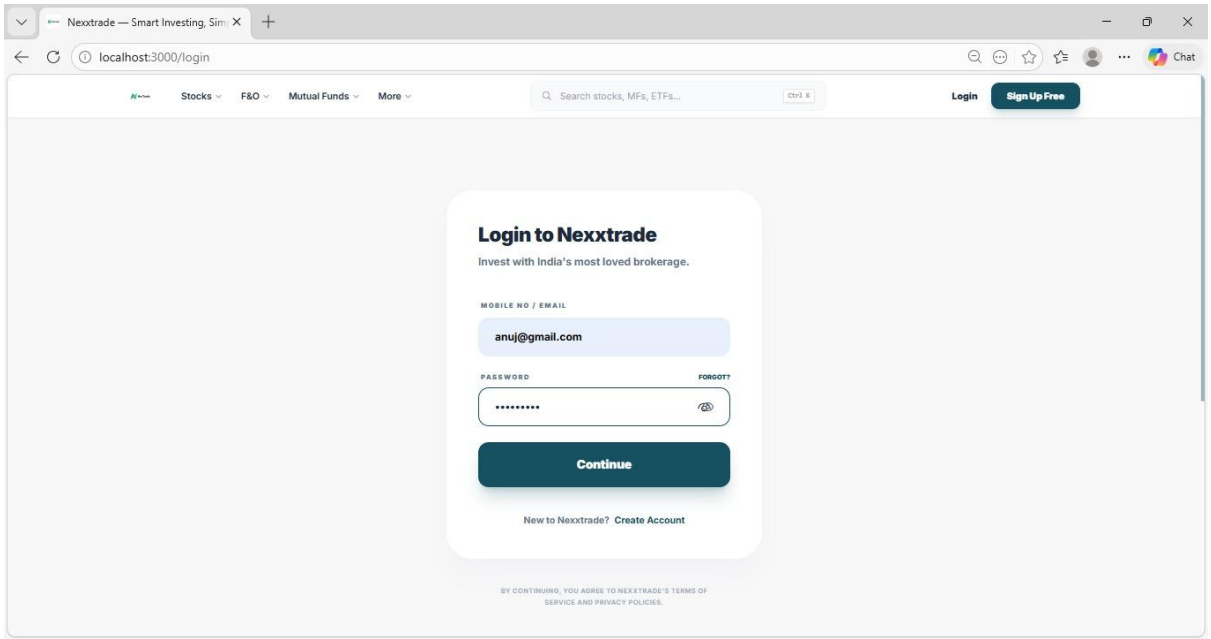
STEP – 9




MongoDB Database Structure – UI/UX & Data Overview

This screen displays the MongoDB Compass dashboard for the stock-trading database within the tradewise connection. The left sidebar navigation shows available databases and collections, enabling quick access to different datasets. In the main panel, the database structure is presented in a tabular layout, listing collections such as orders, transactions, and users. Each collection includes metadata like storage size, number of documents, average document size, and indexes, helping developers monitor database performance and structure. The top action bar provides controls like Create Collection, Refresh, and Open MongoDB Shell, making database management efficient. The layout follows a clear data hierarchy, where collections act as tables and documents represent stored records. This interface helps developers easily analyze, organize, and manage backend data for the stock trading application.

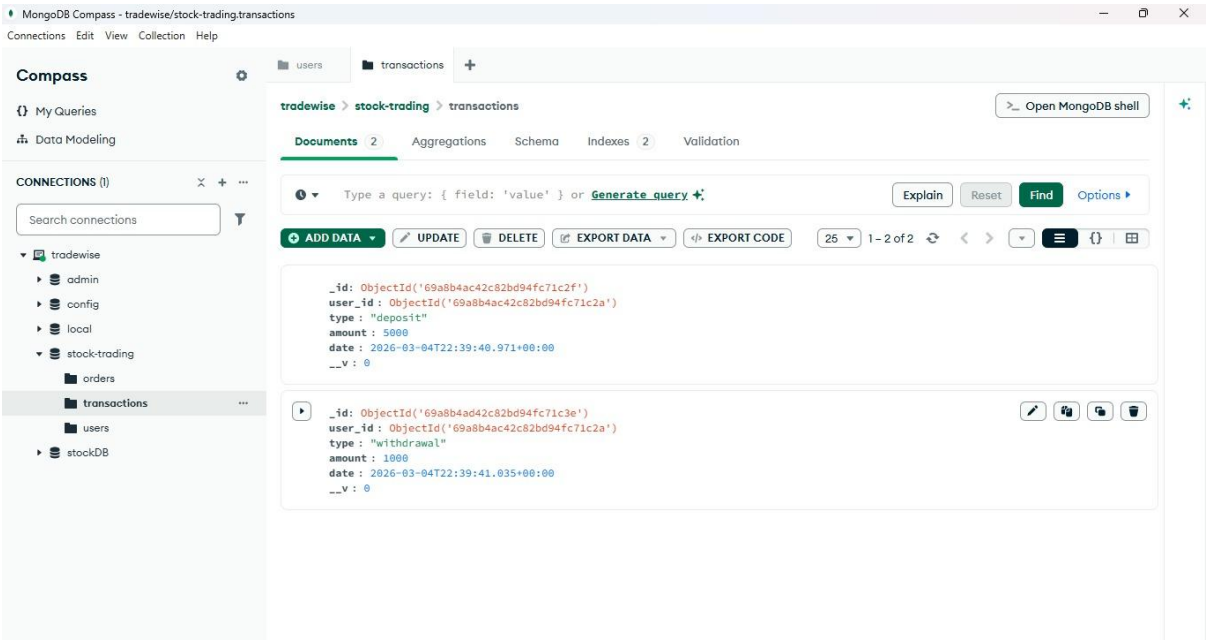
STEP – 10




Login Page – UI/UX Breakdown

This screen shows the login interface of the Nexxtrade investment platform, designed to provide a simple and secure user authentication experience. The top navigation bar remains consistent with the rest of the platform, featuring sections like Stocks, F&O, Mutual Funds, and a search bar for quick access to market assets. At the center of the page, a card-based login form creates focus and improves usability by separating the authentication area from the rest of the layout. The form includes input fields for email/mobile number and password, along with a password visibility toggle and a “Forgot?” option for account recovery. A prominent Continue button acts as the primary call-to-action, guiding users to proceed with login. Below the form, a Create Account link helps new users register quickly. The page uses minimal design, rounded components, soft shadows, and balanced whitespace, ensuring clarity, accessibility, and a modern fintech user experience.

STEP – 11




MongoDB Transactions Collection – UI/UX & Data Overview

This screen shows the MongoDB Compass interface displaying the transactions collection within the stock-trading database. The left sidebar provides a structured view of the database hierarchy, including collections such as orders, transactions, and users for easy navigation. In the main panel, the Documents tab lists transaction records stored in JSON format. Each document represents a financial transaction, containing fields like user_id, type (deposit or withdrawal), amount, and date. These records track how users add or withdraw funds from their trading account. The top action toolbar includes options such as Add Data, Update, Delete, Export Data, and Find, allowing developers to manage and query database records efficiently. The interface follows a clear document-based data structure, making it easy to monitor financial activities within the trading platform backend.